



哈爾濱工業大學 (深圳)
HARBIN INSTITUTE OF TECHNOLOGY

实验设计报告

开课学期: 2024 年秋季

课程名称: 操作系统

实验名称: 页表

实验性质: 课内实验

实验时间: 11.8 地点: T2608

学生班级: 5

学生学号: 220110515

学生姓名: 金正达

评阅教师:

报告成绩:

实验与创新实践教育中心印制

2024 年 9 月

一、 回答问题

1. 查阅资料，简要阐述页表机制为什么会被发明，它有什么好处？

页表是为了解决计算机系统中内存管理的复杂性和效率问题，特别是在多进程环境中如何有效地分配和管理内存。

好处：

操作系统通过页表将虚拟地址映射到物理内存地址，使不同程序之间的内存地址互不干扰，提高了内存使用的安全性和灵活性。

操作系统通过页表来控制进程对内存的访问权限，从而有效防止进程间的非法访问和干扰。

页表机制将内存划分为大小相等的页，避免了物理内存中碎片的产生，提高了内存的利用率。

2. 按照步骤，阐述 SV39 标准下，给定一个 64 位虚拟地址为

0xFFFFF789ABCDEF 的时候，是如何一步一步得到最终的物理地址

的？（页表内容可以自行假设）

虚拟地址划分为：

$VPN[2] + VPN[1] + VPN[0] + Offset$

位宽为：

$9 + 9 + 9 + 12$

1. 虚拟地址 0xFFFFF789ABCDEF 对应的 各项为

$VPN[2] = 111111111$ (511)

$VPN[1] = 111111110$ (510)

$VPN[0] = 011101110$ (470)

$Offset = 0011001101111101101111$ (0xCDF)

2. 根据 $VPN[2] = 511$ ，查找第三级页表中索引为 511 的页表项。

假设第三级页表的第 511 项映射到物理地址 0x10000000。

3. 第三级页表返回的物理地址 0x10000000 对应一个第二级页表。

根据 $VPN[1] = 510$ ，查找第二级页表中的第 510 项。

假设第二级页表的第 510 项映射到物理地址 0x20000000。

4. 第二级页表返回的物理地址 0x20000000 对应一个第一级页表。

根据 $VPN[0] = 470$ ，查找第一级页表中的第 470 项。

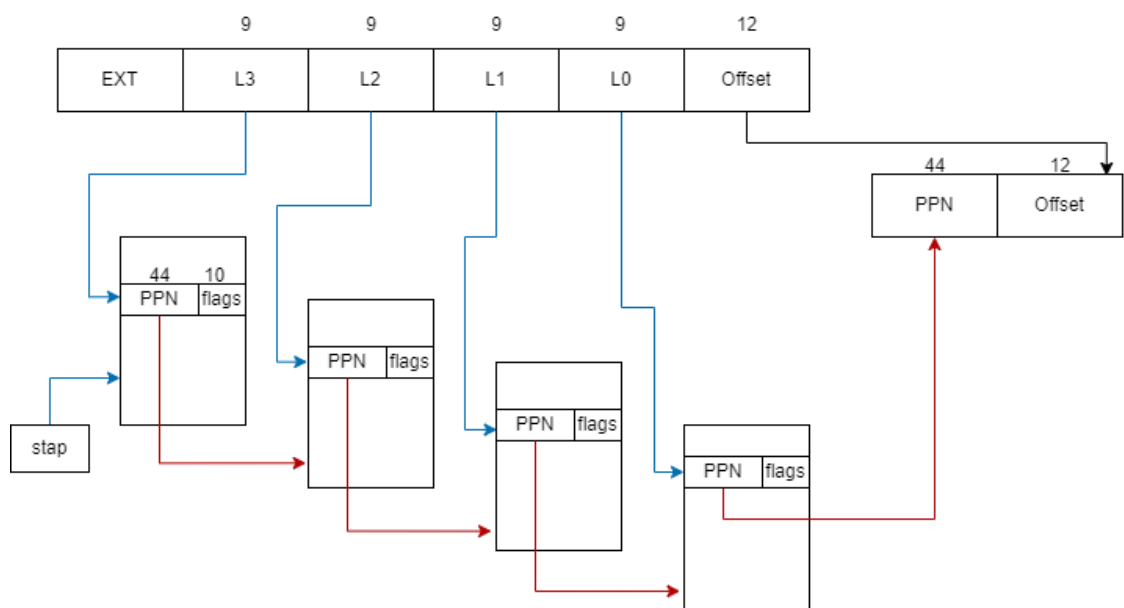
假设第一级页表的第 470 项映射到物理地址 0x30000000。

5. 第一级页表返回的物理地址 0x30000000 加上页内偏移量即为最终物理地址：
 $0x30000000 + 0xCDF = 0x30000CDF$

3. 我们注意到，SV39 标准下虚拟地址的 L2, L1, L0 均为 9 位。这实际上是设计中的必然结果，它们只能是 9 位，不能是 10 位或者是 8 位，你能说出其中的理由吗？（提示：一个页目录的大小必须与页的大小等大）

SV39 标准中，页大小是 4KB，页目录大小也是 4KB，每个页表项 (PTE) 大小是 8B，一个页目录最多容纳 $4KB / 8B = 512$ 个页表项，512 个页表项恰好需要 9 位二进制数来索引。故 9 位的位宽是由页大小和页表项大小共同决定的。

4. 在“实验原理”部分，我们知道 SV39 中的 39 是什么意思。但是其实还有一种标准，叫做 SV48，采用了四级页表而非三级页表，你能模仿“实验原理”部分示意图，画出 SV48 页表的数据结构和翻译的模式图示吗？（[SV39 原图](#)请参考指导书）



二、 实验详细设计

注意不要照搬实验指导书上的内容，请根据你自己的设计方案来填写。

任务一：

采用三层嵌套循环逐层访问三级页表；

逐层判断，pte 存在并且有效时，打印并继续遍历

```
if (pte & PTE_V) {
```

物理地址通过宏 PTE2PA 计算

打印最后一级页表时，通过多级页表索引计算对应的虚拟地址

```
uint64 va = ((uint64)i << 30) | ((uint64)j << 21) | ((uint64)k << 12);
```

任务二：

在 proc.h 的 proc 结构体中增加相关数据结构，用于存储进程内核页表和内核栈：

```
pagetable_t k_pagetable;    // kernel page table
uint64 kstack_pa;           // physical address of kernel stack
```

进程内核页表初始化函数，仿造全局内核页表初始化函数：

```
pagetable_t u_kvmmap(void) {
    pagetable_t pgtbl = (pagetable_t)kalloc();
    memset(pgtbl, 0, PGSIZE);

    // uart registers
    uvmmmap(pgtbl, UART0, UART0, PGSIZE, PTE_R | PTE_W);

    // virtio mmio disk interface
    uvmmmap(pgtbl, VIRTIO0, VIRTIO0, PGSIZE, PTE_R | PTE_W);

    // PLIC
    uvmmmap(pgtbl, PLIC, PLIC, 0x400000, PTE_R | PTE_W);

    // map kernel text executable and read-only.
    uvmmmap(pgtbl, KERNBASE, KERNBASE, (uint64)etext - KERNBASE, PTE_R | PTE_X);

    // map kernel data and the physical RAM we'll make use of.
    uvmmmap(pgtbl, (uint64)etext, (uint64)etext, PHYSTOP - (uint64)etext, PTE_R | PTE_W);

    // map the trampoline for trap entry/exit to
    // the highest virtual address in the kernel.
    uvmmmap(pgtbl, TRAMPOLINE, (uint64)trampoline, PGSIZE, PTE_R | PTE_X);

    return pgtbl;
}
```

不对 CLINT 进行映射，其中 uvmmmap 为映射特定页表的函数：

```
void uvmmmap(pagetable_t pgtbl, uint64 va, uint64 pa, uint64 sz, int perm) {
    if (mappages(pgtbl, va, sz, pa, perm) != 0) panic("uvmmmap");
}
```

在 procinit 函数中，取消将内核栈映射到全局内核页表中的操作，将内核栈保存到进程结构体相应变量的 kstack, kstack_pa 中：

```

void procinit(void) {
    struct proc *p;

    initlock(&pid_lock, "nextpid");
    for (p = proc; p < &proc[NPROC]; p++) {
        initlock(&p->lock, "proc");

        // Allocate a page for the process's kernel stack.
        // Map it high in memory, followed by an invalid
        // guard page.
        char *pa = kalloc();
        if (pa == 0) panic("kalloc");
        uint64 va = KSTACK((int)(p - proc));
        // kvmmap(va, (uint64)pa, PGSIZE, PTE_R | PTE_W);
        p->kstack = va;
        p->kstack_pa = (uint64)pa;
    }
    kvminithart();
}

```

在 `allocproc` 函数中初始化进程的内核页表，并将保存的内核栈映射到其中：

```

p->k_pagetable = u_kvmminit();
if (p->k_pagetable == 0) {
    freeproc(p);
    release(&p->lock);
    return 0;
}
uvmmmap(p->k_pagetable, p->kstack, p->kstack_pa, PGSIZE, PTE_R | PTE_W);

```

在进程调度前，将进程的内核页表加载到 `stap` 寄存器中，调度结束之后，切换回全局内核页表，在 `scheduler` 函数中添加：

```

u_kvminithart(p->k_pagetable);
swtch(&c->context, &p->context);
kvminithart();

```

其中，`u_kvminithart` 函数为 `kvminithart` 函数的用户版本，操作的是传入的页表：

```

void u_kvminithart(pagetable_t pagtbl) {
    w_satp(MAKE_SATP(pagtbl));
    sfence_vma();
}

```

在 `freeproc` 函数中，增加对进程内核页表的回收：

```

if (p->k_pagetable) freekpgtbl(p->k_pagetable);
p->k_pagetable = 0;

```

其中，`freekpgtbl` 函数回收传入的页表，但不释放叶子页表指向的物理帧：

```

void freekpgtbl(pagetable_t pagetable) {
    for (int i = 0; i < 512; i++) {
        pte_t pte = pagetable[i];
        if ((pte & PTE_V) && (pte & (PTE_R | PTE_W | PTE_X)) == 0) {
            uint64 child = PTE2PA(pte);
            freekpgtbl((pagetable_t)child);
            pagetable[i] = 0;
        }
    }
    kfree((void *)pagetable);
}

```

任务三：

`sync_pagetable` 函数将进程的用户页表映射到内核页表中，根据 `va` 逐步遍历，获得一级页表的 `pte`，将用户内核页表的 `pte` 值设置为用户页表的 `pte`，最后再设置标志位，使内核不能访问：

```
void sync_pagetable(pagetable_t u_pgtbl, pagetable_t k_pgtbl, uint64 start, uint64 end) {
    pte_t *u_pte, *k_pte;

    for (uint64 va = start; va < end; va += PGSIZE) {
        u_pte = walk(u_pgtbl, va, 0);
        k_pte = walk(k_pgtbl, va, 1);
        *k_pte = *u_pte;
        *k_pte &= ~(PTE_U|PTE_W|PTE_X);
    }
}
```

exec 函数中，添加映射，由于是舍弃所有页表建立新的页表，所以遍历范围为 0 到 SZ:

```
p->trapframe->sp = sp; // initial stack pointer
proc_freepagetable(oldpagetable, oldsz);

sync_pagetable(pagetable, p->k_pagetable, 0, p->sz);

if (p->pid == 1) {
```

fork 函数中，同样是全部映射，遍历范围为 0 到 sz:

```
safestrcpy(np->name, p->name, sizeof(p->name));

sync_pagetable(np->pagetable, np->k_pagetable, 0, np->sz);

pid = np->pid;
```

growproc 函数中，页表只是范围发生变化，遍历范围为旧的 sz 到新的 sz:

```
sz = p->sz;
if (n > 0) {
    if ((sz = uvmmalloc(p->pagetable, sz, sz + n)) == 0) {
        return -1;
    }
} else if (n < 0) {
    sz = uvmmalloc(p->pagetable, sz, sz + n);
}
sync_pagetable(p->pagetable, p->k_pagetable, p->sz, sz);
p->sz = sz;
return 0;
```

userinit 函数中，是全部映射，遍历范围为 0 到 sz:

```
safestrcpy(p->name, "initcode", sizeof(p->name));
p->cwd = namei("/");

sync_pagetable(p->pagetable, p->k_pagetable, 0, p->sz);

p->state = RUNNABLE;
```

三、实验结果截图

请给出任务一、任务二、任务三和 make grade 的运行结果截图。

任务一:

```
hart 2 starting
hart 1 starting
page table 0x0000000087f25000
||idx: 0: pa: 0x0000000087f21000, flags: ----
||  ||idx: 0: pa: 0x0000000087f20000, flags: ----
||  ||  ||idx: 0: va: 0x0000000000000000 -> pa: 0x0000000087f20000, flags: rwxu
||  ||  ||idx: 1: va: 0x0000000000001000 -> pa: 0x0000000087f1f000, flags: rwxu
||  ||  ||idx: 2: va: 0x0000000000002000 -> pa: 0x0000000087f1e000, flags: rwxu
||idx: 255: pa: 0x0000000087f24000, flags: ----
||  ||idx: 511: pa: 0x0000000087f23000, flags: ----
||  ||  ||idx: 510: va: 0x00000003fffffe000 -> pa: 0x0000000087f76000, flags: rwxu
||  ||  ||idx: 511: va: 0x00000003fffffe000 -> pa: 0x0000000080007000, flags: rwxu
init: starting sh
$
```

任务二：

```
$ kvmtest
kvmtest: start
test_pagetable: 1
kvmtest: OK
```

任务三：

```
$ stats stats
copyin: 58
copyinstr: 12
```

usertests:

```
test mem: OK
test pipe1: OK
test preempt: kill... wait... OK
test exitwait: OK
test rmdot: OK
test fourteen: OK
test bigfile: OK
test dirfile: OK
test iref: OK
test forktest: OK
test bigdir: OK
ALL TESTS PASSED
```

make grade:

```
== Test pte printout ==
$ make qemu-gdb
pte printout: OK (2.2s)
== Test count copyin ==
$ make qemu-gdb
count copyin: OK (0.7s)
== Test kernel pagetable test ==
$ make qemu-gdb
kernel pagetable test: OK (1.0s)
== Test usertests ==
$ make qemu-gdb
(78.7s)
== Test usertests: copyin ==
usertests: copyin: OK
== Test usertests: copyinstr1 ==
usertests: copyinstr1: OK
== Test usertests: copyinstr2 ==
usertests: copyinstr2: OK
== Test usertests: copyinstr3 ==
usertests: copyinstr3: OK
== Test usertests: sbrkmuch ==
usertests: sbrkmuch: OK
== Test usertests: all tests ==
usertests: all tests: OK
Score: 100/100
```

四、 实验总结

请总结 xv6 4 个实验的收获，给出对 xv6 实验内容的建议。

注：本节为酌情加分项。

收获：

了解熟悉了 xv6 的启动流程、用户程序的调用过程，学会使用 gdb 对系统代码进行调试。

通过实验中的 syscall 实现，熟悉了操作系统如何将用户程序的请求映射到内核。

学会了如何设置分配锁来完成原子性操作，学会了如何设计锁来保证访问安全和尽量避免死锁。

通过 pgtbl 实验，了解了页表和虚拟内存的基本组织形式和管理方式，学会了索引页表结构进行调试。

建议：

可以增加一些多核下的并发编程实验，帮助理解程序如何在多个核心上调度，以及程序如何在多个核心上进行有效的同步与资源管理。

可以增加更详细的调试提示，在实验过程中，调试和跟踪错误很困难，尤其是涉及到汇编代码时，xv6 的报错信息也是很难看得懂，往往需要花费大量时间来定位问题。